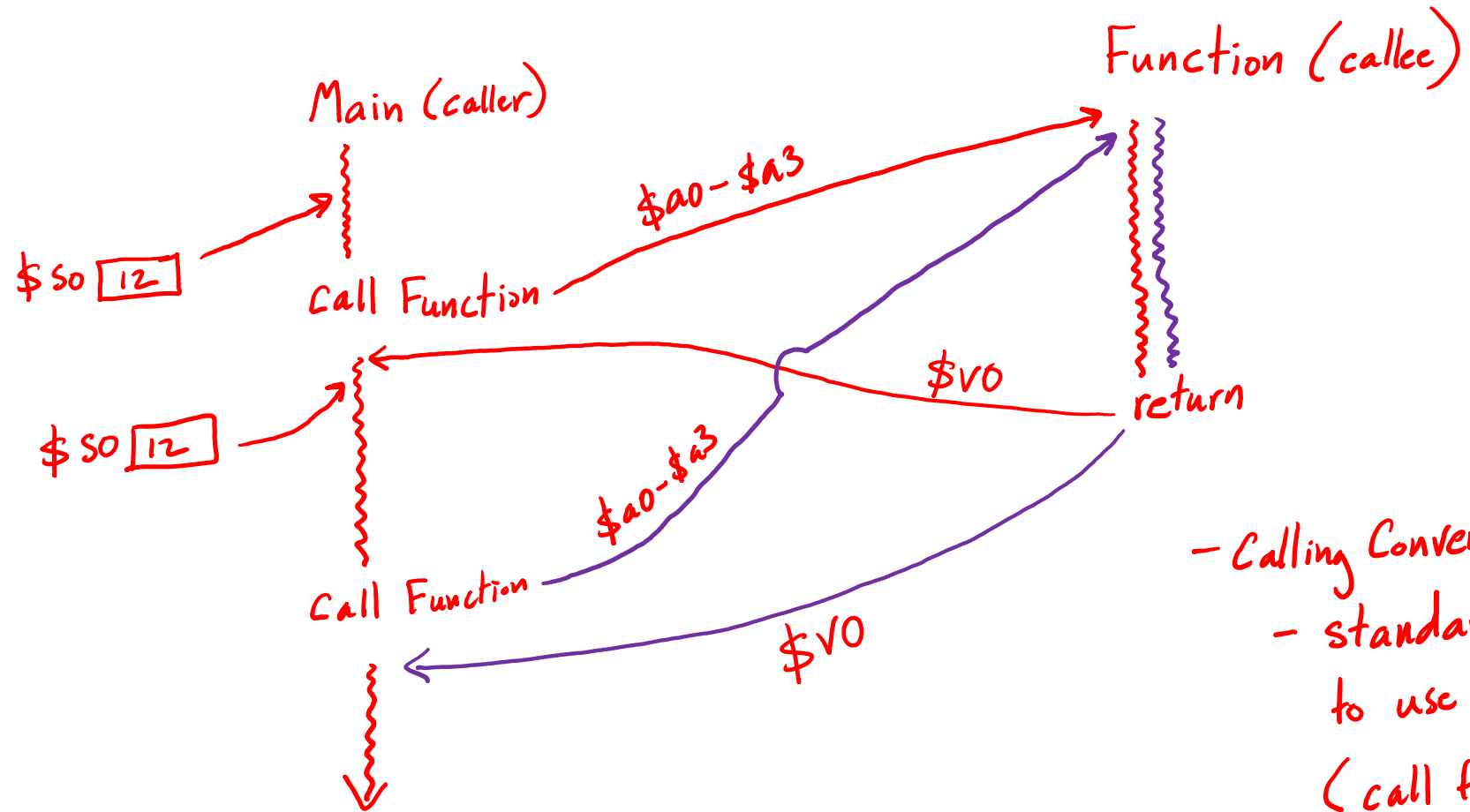


MIPS Calling Convention

- Chapter 2.8

Procedure Calls



- Calling Convention
- standard way to use registers (call func, pass arg., get return values)

MIPS-specific info

Page 121

Name	Reg Number	Usage	Preserved across call?
\$zero, \$0	0	The constant 0	Yes
<i>return values</i> → \$v0-\$v1	2-3	Function results	No
<i>arguments</i> → \$a0-\$a3	4-7	Function Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved	Yes
\$t8-\$t9	24-25	More temporaries	No
\$gp	28	Global pointer	Yes
\$sp	29	Stack pointer	Yes
\$fp	30	Frame pointer	Yes
\$ra	31	Return address	Yes

MIPS-specific info

Preserved - value is the same after a call

Not Preserved - value may change during the function call (and after)

Steps for caller (Main)

1. put arguments into $\$a0 - \$a3$

2. call function $\longrightarrow$ using jal:

3. find the return value
in $\$v0$

1. jumps

2. puts $PC + 4$
into the $\$ra$
register

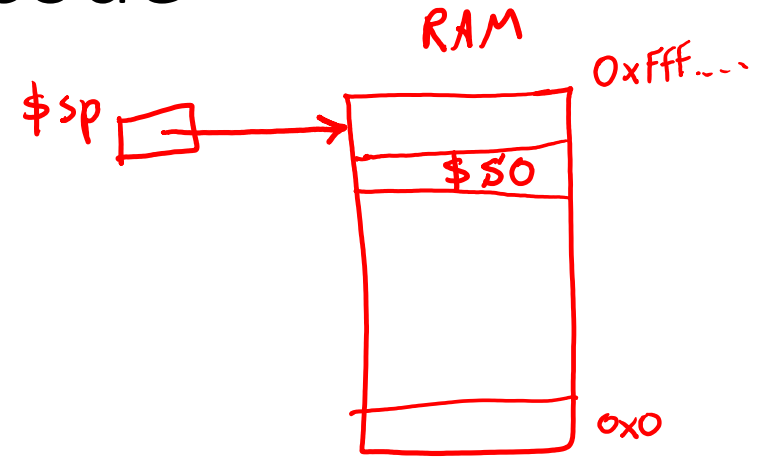
address of
the next inst.

Steps for procedure (callee)

1. store any preserved registers that will be modified (save on the stack)
2. do computation
3. restore preserved reg.
4. put answer into \$v0
5. return using $\rightarrow$ jr \$ra

Write the caller & procedure code for the following function:

```
int MyFunc(int $a0g, int $a1h)
{ return (g + h); }
```



(Main) Caller:

put args. g, h $\$s2, \$s3$

```
add $a0, $s2, $0
add $a1, $s3, $0
jal MyFunc
add $s0, $v0, $0
```

$\$v0$

(MyFunc) Callee:

push $\$s0$ on the stack

```
addi $sp, $sp, -4
sw $s0, 0($sp)
add $s0, $a0, $a1
add $v0, $s0, $0
lw $s0, 0($sp)
addi $sp, $sp, 4
jr $ra
```

pop $\$s0$ off stack

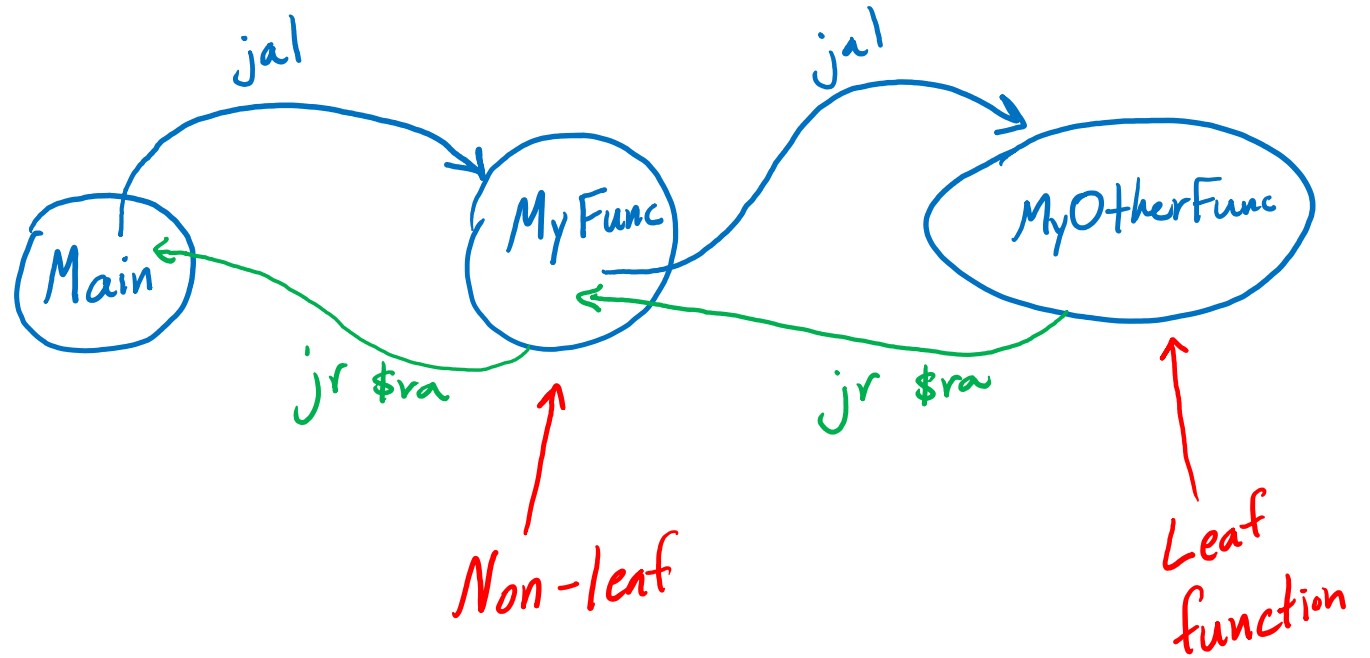
puts return value into $\$v0$

Assume: g, h are in $\$s2, \$s3$. We want return value in $\$s0$.

Preserved Registers

Name	Reg Number	Usage	Preserved across call?
\$zero	0	The constant 0	Yes
\$v0-\$v1	2-3	Function results	No
\$a0-\$a3	4-7	Function Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved	Yes
\$t8-\$t9	24-25	More temporaries	No
\$gp	28	Global pointer	Yes
\$sp	29	Stack pointer	Yes
\$fp	30	Frame pointer	Yes
\$ra	31	Return address	Yes

Definitions



- Leaf Function
 - does not call other function

- Non-leaf
 - a function that calls other functions

- for non-leaf functions,
save the \$ra register
to the stack
(so that MyFunc can return to Main)

~~How much stack space?~~

Lab 1
Question 3

64

0000.....001

00.....001

65536

= X Y

65536 = X, Y

Allocating stack space

